

System Design — Complete Course Notes

1. What is System Design?

System Design is not about memorizing fancy terminology — it's about learning **how to think** while building applications that can serve millions of users reliably. The course builds this mindset using a running example: the "**Alien Bank**" — a hypothetical banking application used to illustrate how a system evolves from a simple idea into a distributed, production-grade architecture.

Key idea: every real system is a series of trade-offs. As you add more users, more data, and more features, a single server and a single database stop being sufficient, and you must consciously choose between consistency, availability, cost, and complexity.

Core Components of a System

Almost every large-scale system is built from the same building blocks:

- **Client Applications** — web, mobile, or desktop apps that users interact with.
 - **Backend Servers** — process business logic and respond to client requests.
 - **Databases** — store and retrieve persistent data.
 - **Load Balancers** — distribute incoming traffic across multiple servers.
 - **Message Queues** — decouple services and enable asynchronous processing.
 - **Monitoring & Logs** — give visibility into system health and behavior.
-

2. Types of Applications

Data-Intensive Applications

These applications are bottlenecked by the **volume, variety, and velocity of data** rather than raw computation. Example: Instagram — storing and serving billions of photos/videos, handling feeds, likes, comments, and metadata at massive scale. The challenge is storage, retrieval speed, and consistency across huge datasets.

Compute-Intensive Applications

These are bottlenecked by **processing power**, not data volume. Examples include Machine Learning training/inference and rendering workloads. These are typically **CPU-bound or GPU-bound**, and scaling them means adding more powerful or more parallel compute resources rather than just more storage.

Understanding whether your system leans data-intensive or compute-intensive shapes almost every downstream architecture decision (choice of database, caching strategy,

scaling strategy).

3. Functional vs Non-Functional Requirements

- **Functional Requirements** — what the system should *do* (e.g., "users can transfer money," "users can upload videos").
- **Non-Functional Requirements** — how well the system should do it:
 - **Scalability** — ability to handle growing load.
 - **Availability** — system remains operational and accessible.
 - **Reliability** — system performs correctly and consistently over time.
 - **Performance** — speed and responsiveness under load.
 - **Security** — protection against unauthorized access/attacks.
 - **Maintainability** — ease of updating and debugging the system.
 - **Observability** — ability to understand internal system state from external outputs (logs, metrics, traces).

Every design decision later in the course maps back to satisfying one or more of these non-functional requirements.

4. DNS (Domain Name System)

DNS is how a human-readable domain name (e.g., `example.com`) gets translated into a machine-usable IP address. The resolution process moves through a hierarchy:

1. **Root Server** — top of the DNS hierarchy, points to TLD servers.
2. **TLD (Top-Level Domain) Server** — handles domains like `.com`, `.org`, `.in`.
3. **Name Server (Authoritative Server)** — holds the actual IP address records for the domain.

DNS Resolution Steps (simplified): Browser → checks cache → OS resolver → Root Server → TLD Server → Authoritative Name Server → returns IP → browser connects to that IP.

This step happens before almost every network request and is often the first hidden layer of "system design" a user never sees.

5. APIs and Communication

An **API (Application Programming Interface)** defines how two systems communicate. Different API styles suit different needs:

API Type	Best Suited For
REST	Standard web/mobile CRUD apps, resource-based access
SOAP	Enterprise systems needing strict contracts/security (banking, legacy systems)
GraphQL	Clients needing flexible, precise data fetching (avoiding over/under-fetching)
gRPC	High-performance, low-latency service-to-service communication (microservices)
WebSockets	Real-time, bidirectional communication (chat apps, live notifications)

REST API Deep Dive

- **HTTP Methods:** GET (read), POST (create), PUT (full update), PATCH (partial update), DELETE (remove).
- **Endpoints** represent resources (e.g., `/users/123`).
- **JSON** is the standard payload format for requests/responses.
- **Query Parameters** filter/modify a request (e.g., `?sort=asc`).
- **Request/Response Body** carries the actual data payload.
- **Status Codes** communicate outcome: 2xx success, 3xx redirection, 4xx client error, 5xx server error.
- **API Design Best Practices:** consistent naming, proper status codes, versioning, pagination, and clear error messages.

6. Databases

SQL (Relational Databases)

Data is organized into **tables** with **rows** and **columns**, enforced by a strict schema.

- **Constraints:** rules like NOT NULL, UNIQUE, CHECK that maintain data integrity.
- **Primary Key:** uniquely identifies each row.
- **Foreign Key:** links a row to a row in another table, enforcing relationships.
- **Joins:** combine data across tables — used to implement:
 - **One-to-One** relationships
 - **One-to-Many** relationships
 - **Many-to-Many** relationships (via junction tables)

SQL databases are ideal when data is structured, relationships matter, and strong consistency (ACID) is required.

NoSQL (Non-Relational Databases)

As applications scale, rigid schemas and single-server relational databases become bottlenecks. NoSQL databases were built to solve **scaling challenges** by relaxing strict schema and consistency requirements. Types include:

- **Key-Value Databases** (e.g., Redis) — simplest form, fast lookups by key.
- **Document Databases** (e.g., MongoDB) — store flexible, JSON-like documents.
- **Graph Databases** (e.g., Neo4j) — optimized for relationship-heavy data (social networks).
- **Columnar Databases** (e.g., Cassandra) — optimized for fast reads/writes across huge datasets, common in analytics.

SQL vs NoSQL boils down to: structured vs flexible schema, vertical vs horizontal scaling ease, and strong vs eventual consistency.

7. Caching

Caching stores frequently accessed data closer to the application to reduce database load and latency.

- **Cache Hit** — data found in cache (fast). **Cache Miss** — data not found, must fetch from source (slow).
- **TTL (Time To Live)** — how long cached data stays valid before expiry.

Cache Strategies

- **Read-Through Cache** — cache sits between app and DB; on a miss, the cache itself fetches and stores the data.
- **Write-Through Cache** — writes go to cache and DB simultaneously, keeping them in sync.
- **Write-Around Cache** — writes go directly to DB, bypassing cache (cache populated only on reads).
- **Write-Back Cache** — writes go to cache first and are asynchronously flushed to DB later (fast writes, risk of data loss).

Cache Eviction Policies

When the cache is full, an eviction policy decides what to remove:

- **LRU (Least Recently Used)**
- **LFU (Least Frequently Used)**
- **FIFO (First In, First Out)**
- **MRU (Most Recently Used)**

- **LIFO** (Last In, First Out)
-

8. Load Balancers

A load balancer distributes incoming traffic across multiple backend servers so no single server is overwhelmed, improving both availability and performance.

Load Balancing Algorithms

- **Round Robin** — requests distributed sequentially across servers.
- **Least Connections** — request goes to the server with the fewest active connections.
- **Weighted Round Robin** — servers with higher capacity get proportionally more requests.
- **IP Hash** — client IP determines which server handles the request (useful for session persistence).
- **Geo-Based Routing** — routes requests to the server nearest the user's location.

Health Checks

Load balancers continuously ping backend servers to detect failures and automatically stop routing traffic to unhealthy nodes, improving fault tolerance.

9. Distributed Databases

Replication

Copying data across multiple database nodes for redundancy and read scalability.

- **Single Leader Replication** — one primary node handles writes; replicas handle reads.
- **Multi-Leader Replication** — multiple nodes accept writes, then sync (more complex conflict resolution).
- **Leaderless Replication** — any node can accept reads/writes (e.g., Dynamo-style systems).
- **Sync vs Async Replication** — sync guarantees consistency but adds latency; async is faster but risks temporary inconsistency.
- **Quorum** — minimum number of nodes that must acknowledge a read/write for it to be considered successful, balancing consistency and availability.

Partitioning (Sharding)

Splitting a large dataset across multiple database nodes so no single node stores all the data.

- **Partition by Key** — data split based on a chosen key range.

- **Partition by Hash** — a hash function determines which shard stores the data (better distribution).
- **Secondary Index Partitioning** — enables efficient querying on non-primary-key fields across shards.
- **Hotspots** — a challenge where uneven data/access distribution overloads specific shards; mitigated via better key/hash design.

CAP Theorem

In a distributed system, you can only fully guarantee **two out of three** of:

- **Consistency** — every read gets the latest write.
- **Availability** — every request gets a response (even if not the latest data).
- **Partition Tolerance** — system keeps working despite network failures between nodes.

Since network partitions are unavoidable in distributed systems, real designs typically choose between **CP** (consistency-favoring, e.g., banking systems) and **AP** (availability-favoring, e.g., social media feeds).

10. Message Queues

Message queues enable **asynchronous, decoupled communication** between services — a producer sends a message without waiting for the consumer to process it immediately.

- **Sync vs Async Processing** — sync blocks until a response is ready; async lets the sender move on immediately.
 - **FIFO Queue** — messages processed strictly in the order received.
 - **Priority Queue** — higher-priority messages are processed first, regardless of arrival order.
 - **Pub/Sub Model** — publishers broadcast messages to a topic; multiple subscribers receive and process them independently.
 - **Push vs Pull Model** — in push, the queue sends messages to consumers; in pull, consumers actively fetch messages when ready.
 - **Poison Messages** — malformed/unprocessable messages that can repeatedly fail and block a queue; handled via dead-letter queues.
 - **Duplicate Messages** — can occur due to retries; systems must be designed to be idempotent to handle them safely.
-

11. Fault Tolerance

Real systems must be designed assuming failure is inevitable, not exceptional:

- **Hardware Failures** — disk crashes, server outages.
- **Software Failures** — bugs, memory leaks, crashes.
- **Human Errors** — misconfiguration, accidental deletions.

Recovery mechanisms (replication, retries, backups, failover) are what turn these inevitable failures into non-events for end users, directly supporting the **reliability** and **availability** non-functional requirements from Section 3.

12. Monitoring and Observability

You cannot improve what you cannot measure. Monitoring provides visibility into system health:

API Monitoring

- **Throughput** — number of requests handled per unit time.
- **Latency** — time taken to respond to a request.
- **Error Rates** — percentage of failed requests.
- **Health Checks** — automated checks confirming a service is up and functioning.

Machine Monitoring

- **CPU Usage**
- **Memory Usage**
- **Disk I/O**
- **Network I/O**

Together, API-level and machine-level monitoring let engineers detect degradation before it becomes an outage, and provide the data needed for on-call debugging.

13. Final Case Study: System Design of a Streaming Application

The course closes by applying everything above to a real-world design problem: a **video streaming platform** (à la YouTube/Netflix). Key discussion points:

- **How to think while designing systems** — starting from requirements, not jumping straight to technology choices.
- **Video Processing Architecture** — how uploaded videos are transcoded into multiple resolutions/formats for adaptive streaming.
- **Scalability** — using CDNs, load balancers, and distributed storage to serve video at massive scale.

- **Trade-offs** — balancing storage cost, latency, and consistency (e.g., eventual consistency for view counts vs strong consistency for billing/payments).
 - **Diagram Design** — practicing how to communicate an architecture visually and verbally, a critical skill for system design interviews.
-

Quick-Reference Summary Table

Topic	Key Takeaway
System Design Mindset	Trade-offs over memorization
Data vs Compute Intensive	Determines scaling strategy
Non-Functional Requirements	Scalability, Availability, Reliability, Performance, Security, Maintainability, Observability
DNS	Root → TLD → Name Server resolution
APIs	REST, SOAP, GraphQL, gRPC, WebSockets — pick per use case
SQL vs NoSQL	Structure & consistency vs flexibility & scale
Caching	Read/Write-through, Write-around, Write-back + eviction policies
Load Balancing	Round Robin, Least Connections, Weighted, IP Hash, Geo-based
Replication & Partitioning	Redundancy vs distribution of data
CAP Theorem	Pick 2 of Consistency, Availability, Partition Tolerance
Message Queues	Decoupled, async communication (FIFO, Priority, Pub/Sub)
Fault Tolerance	Design for inevitable failure
Monitoring	Throughput, Latency, Errors, Resource usage
Case Study	Streaming platform ties all concepts together

Notes compiled for personal reference and repository documentation.